Announcements

Topic 7: Binary Search Trees

By: Professor Lynam

General Trees

- Definition: *General Tree*
- *A general tree T is empty or is a finite set of Nodes N such that there is a specially-designated root node r* ∈ N with the remaining N – r nodes partitioned into $m \geq 0$ *disjoint subtrees of T*
- Binary Search Trees ⊆ Binary Trees ⊆ General Trees
- Note that this is a recursive definition, meaning you can have sub-trees, and trees being forming by attaching existing trees
- A node in a general tree may have any non-negative quantity of "children"

Binary Trees

- Definition: *Binary Tree*
- *A binary tree is a general tree such that* $0 \leq m \leq 2$.

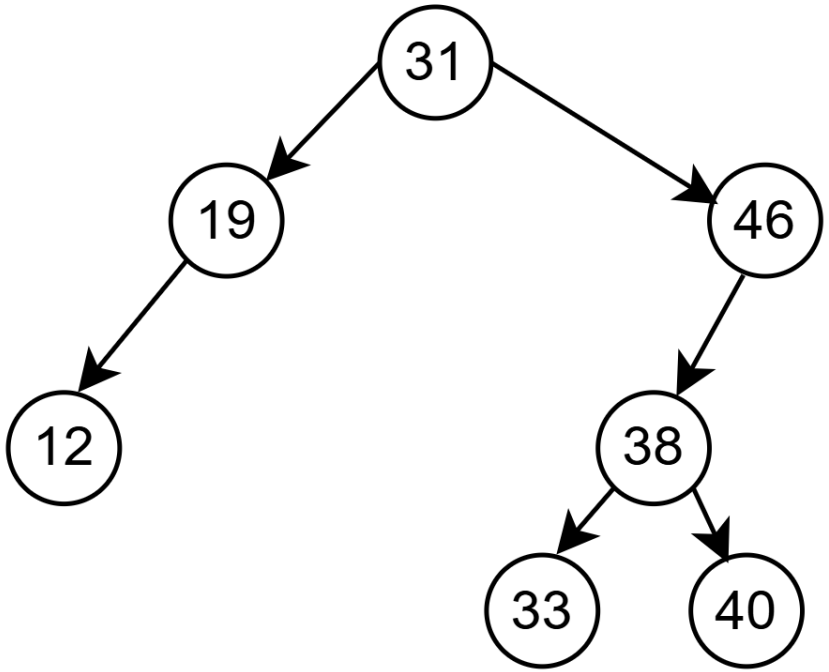
- We call the node "higher" in the tree the parent node, and assuming it has two children, it has a "left" and "right" child node
- Example applications of binary trees include:
- Binary Space Partitioning (BSP) in video games
- Huffman Coding (lossless data compression)
- Heaps (priority queues)
- Expression Trees (intermediate expression representation)
- And last, but not least…

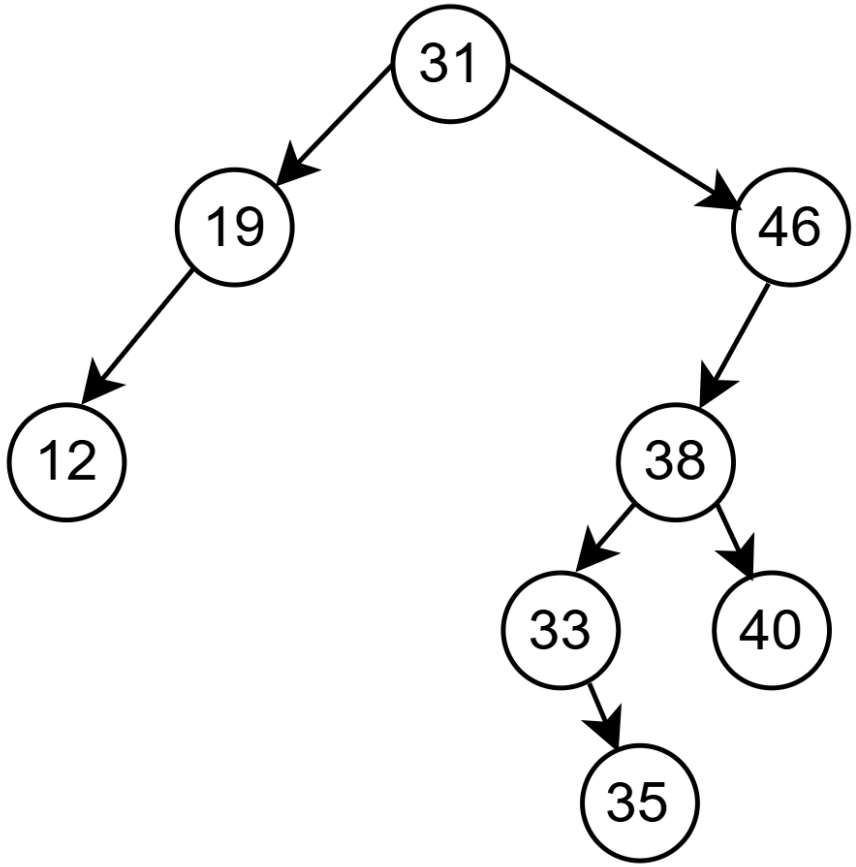
Binary Search Trees (BSTs)

- Definition: *Binary Search Tree*
- *A binary search tree (BST) is a binary tree in which any key value held in a node is greater than any key in that node's left subtree and is less than any key in its right subtree.*

- What about equal key values, you might ask?
- We can extend the definition to store ties on one side or the other (but not both!) of existing matches. (However, in many BST applications, data values are unique.)

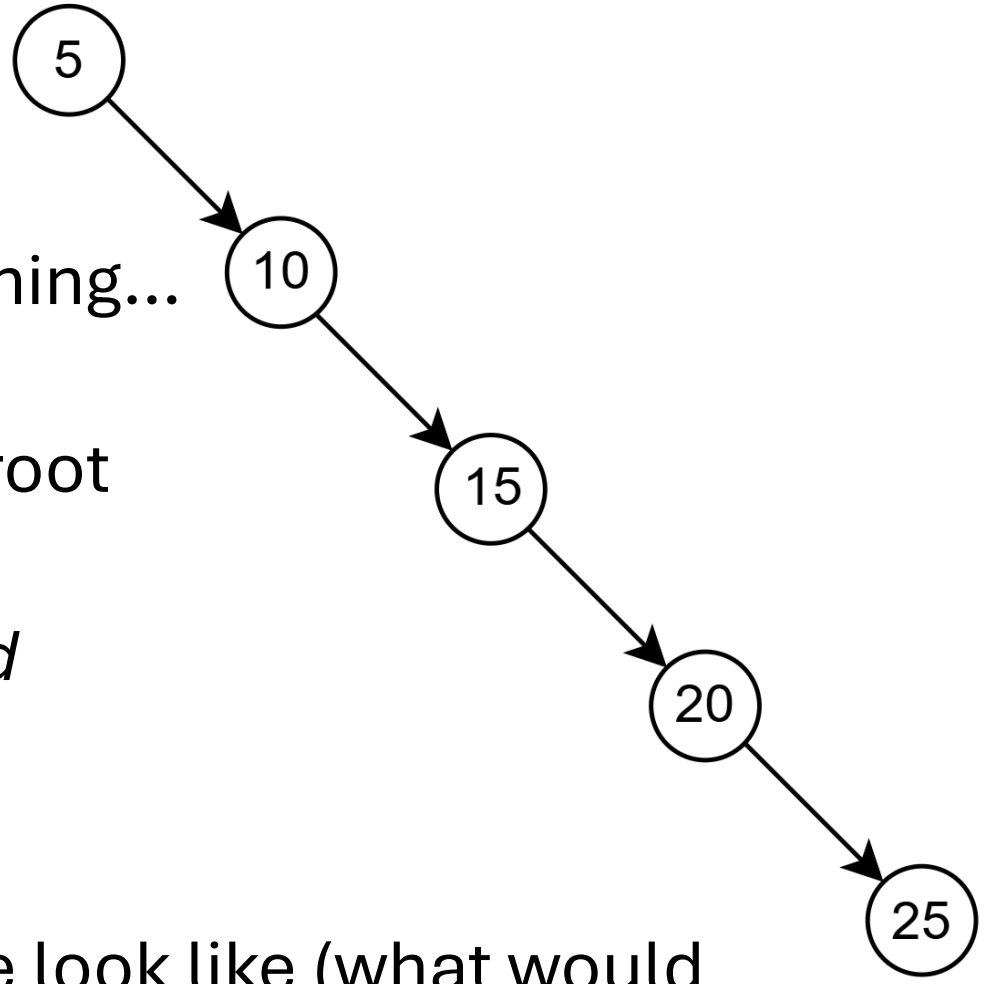
BST Insertions





- Let's insert 35 into this BST. What are the steps?
- Just search for 35, and insert into the fringe where the search fails

BST Searching



- A key advantage of BSTs is efficient searching… let's see how efficient it actually is
- Best case: O(1), when the target is in the root node
- Worst case: O(n), when the tree is *skewed*
- What does a skewed BST look like?
- A Linked List (searching an LL is O(n) too)
- What would the opposite of a skewed tree look like (what would our *ideal* tree look like?)?
- A "full" or "complete" tree, with each level filled
- Average case: depends on the tree shape! Could be O(*h*) to O(n/2)

BST Searching

- Definition: *Complete Tree*
- *A complete tree of height h, where the root is h = 0, has all positions on all levels filled with data.*

- So, what's the capacity of a height *h* binary tree?

Height	Capacity
0	1
1	3 (1 + 2)
2	7 (3 + 4)
3	15 (7 + 8)
…	…
h	$2^{h+1} - 1 (= n)$

BST Searching

- The height of the BST correlates with the number of comparsions necessary for searching… So what is *h* in terms of *n* (the quantity of data)?
- $2^{h+1} - 1 = n$
- $2^{h+1} = n + 1$
- $\log_2 2^{h+1} = \log_2(n + 1)$
- $h = \log_2(n + 1) - 1$
- So h is O($\log_2 n$)
- But most BST aren't complete. Suppose one is just half-full… All we'd have to do is replace n with n/2 in our equation. We'd still get a $\log_2 n$ term

Tree Sort

- The idea for tree sort is that an inorder traversal of a BST visits the keys in ascending order
- The algorithm outline is simple:

1. Build a BST
2. ‘Inorderly’ traverse the BST

Tree Sort

- Analysis time! What's our best case scenario?
- The tree will end up being complete!

1. Build a BST: n insertions at O($\log_2 n$) each -> O($\mathrm{n} * \log_2 n$)
2. 'Inorderly' traverse the BST: T(n) = T(n/2) + 1 + T(n/2)… which is?

- O(n) by the master theorem! So overall, O($\mathrm{n} * \log_2 n$)
- Worst case? We get a completely skewed tree!

1. Build a BST: 1 + 2 + 3 + … + n -> O(n^2)
2. Traverse: still O(n)

- Overall, O(n^2)
- Average case? Same as best case: O($\mathrm{n} * \log_2 n$)

AVL Trees: A Height-Balanced BST

- Definition: *AVL Tree*
- *An AVL tree is a BST satisfying this balance condition: For each node in the tree, the height of its left subtree differs from the height of its right subtree by no more than one.*

- The name is for "Adelson-Velsky and Landis," who invented them in 1962
- If this is a height-balanced BST, do weight-balanced trees exist?
- Yes! Huffman coding trees are one such example.